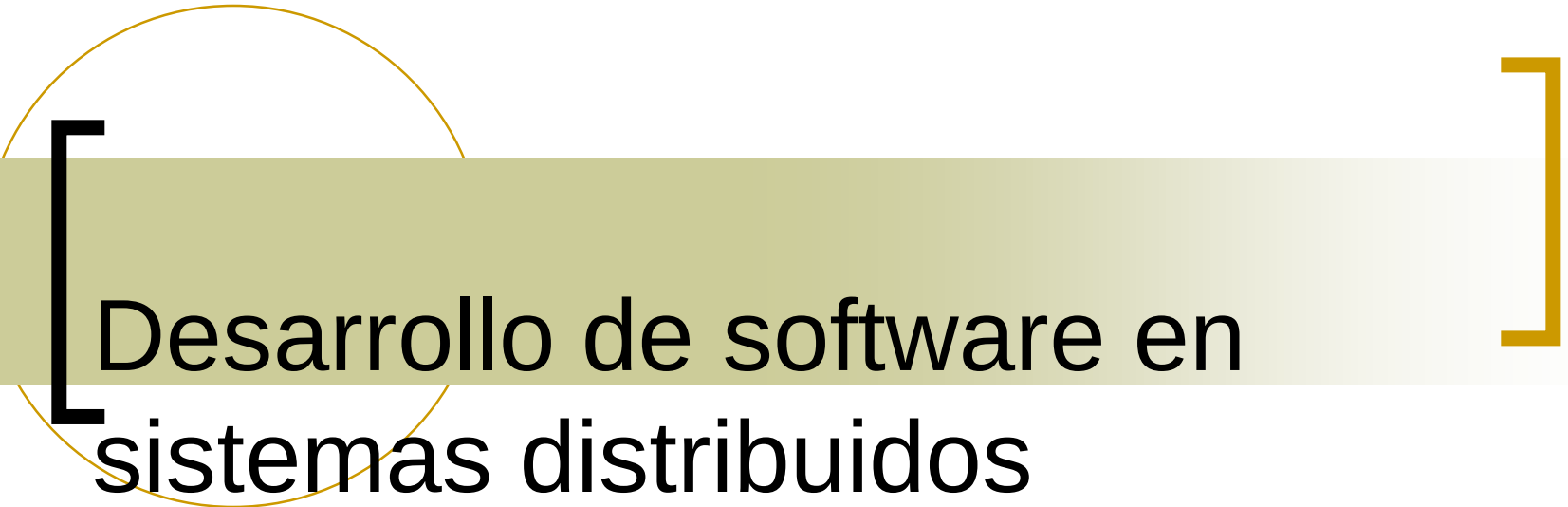




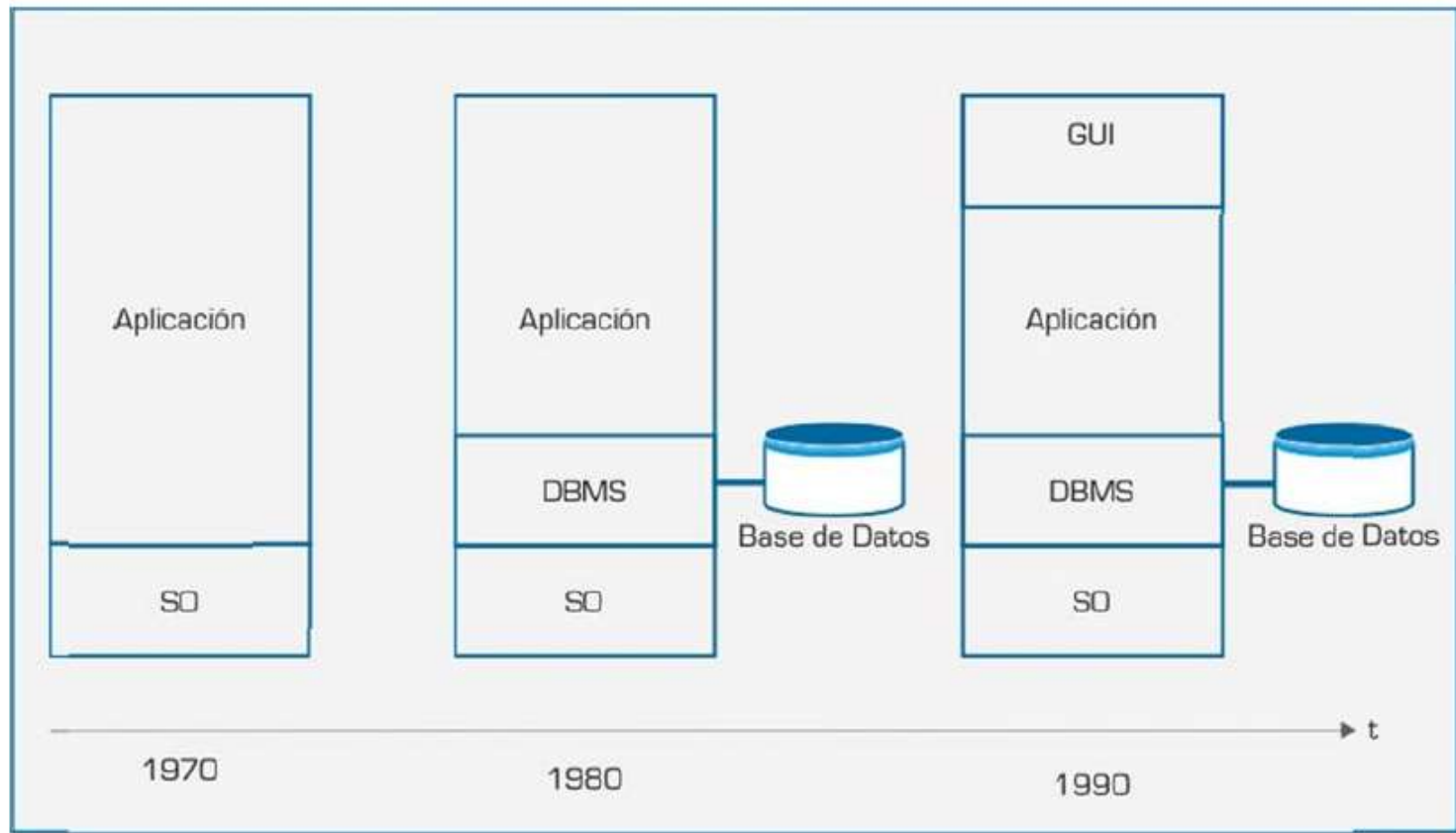
Desarrollo de software en sistemas distribuidos

Curso 2023

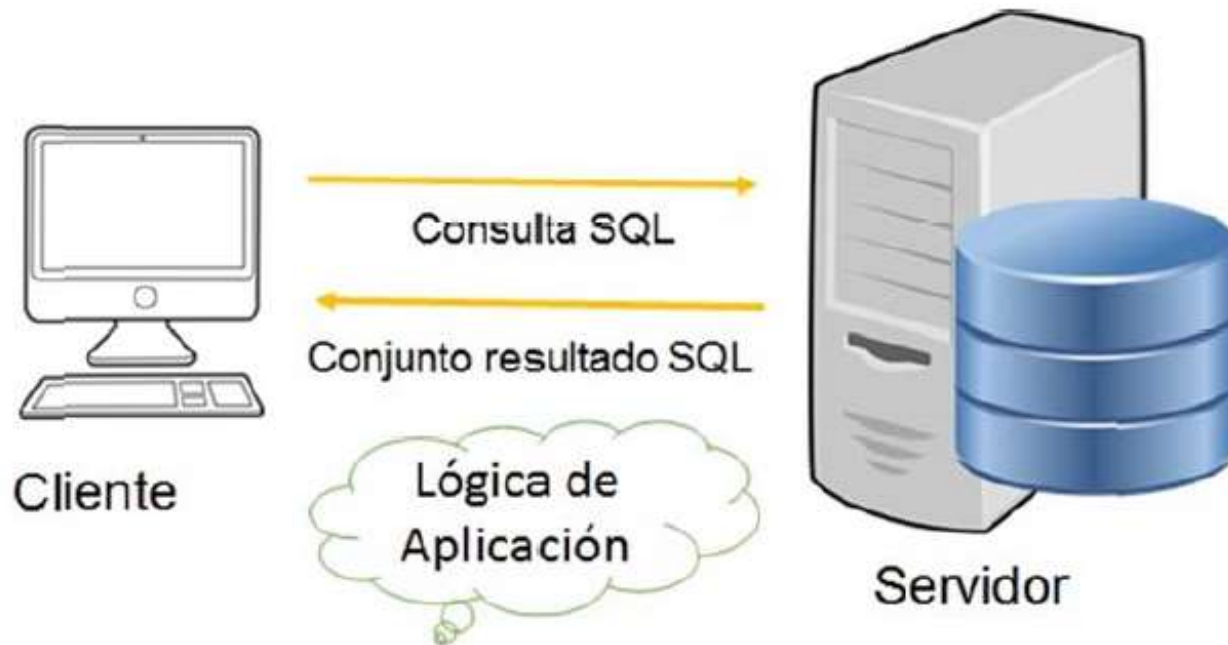
Evolución de los Sistemas de Información

- Arquitectura Monolítica
- Arquitectura Distribuida (2 capas y N capas)
- Arquitecturas Distribuidas con objetos (estándar CORBA)
- Integración de aplicaciones
- Arquitecturas Orientadas a Servicios

Evolución de los Sistemas de Información



Arquitectura Distribuida en 2 Capas



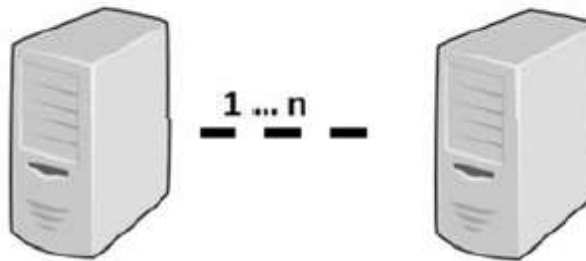
Arquitectura Distribuida en N Capas

Capa Presentación



Cliente

Capa Lógica de Negocio

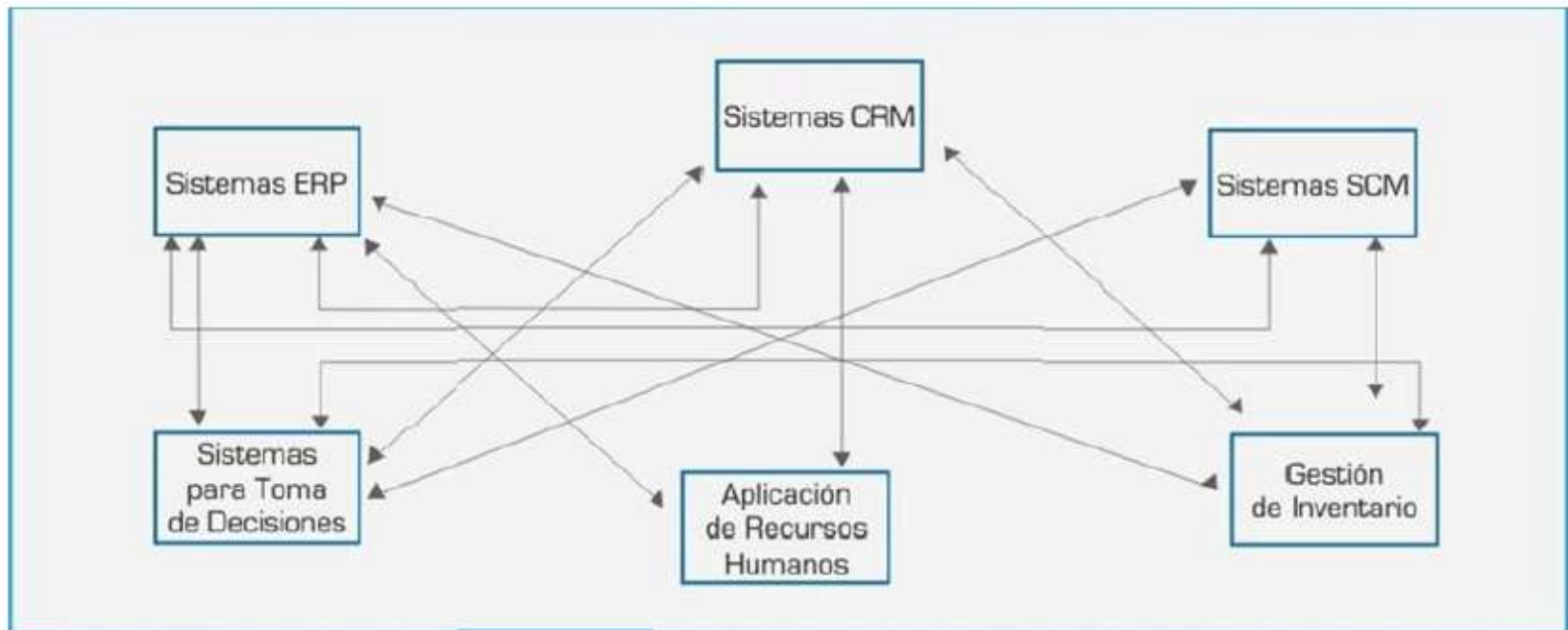


Capa Acceso a Datos



Servidor

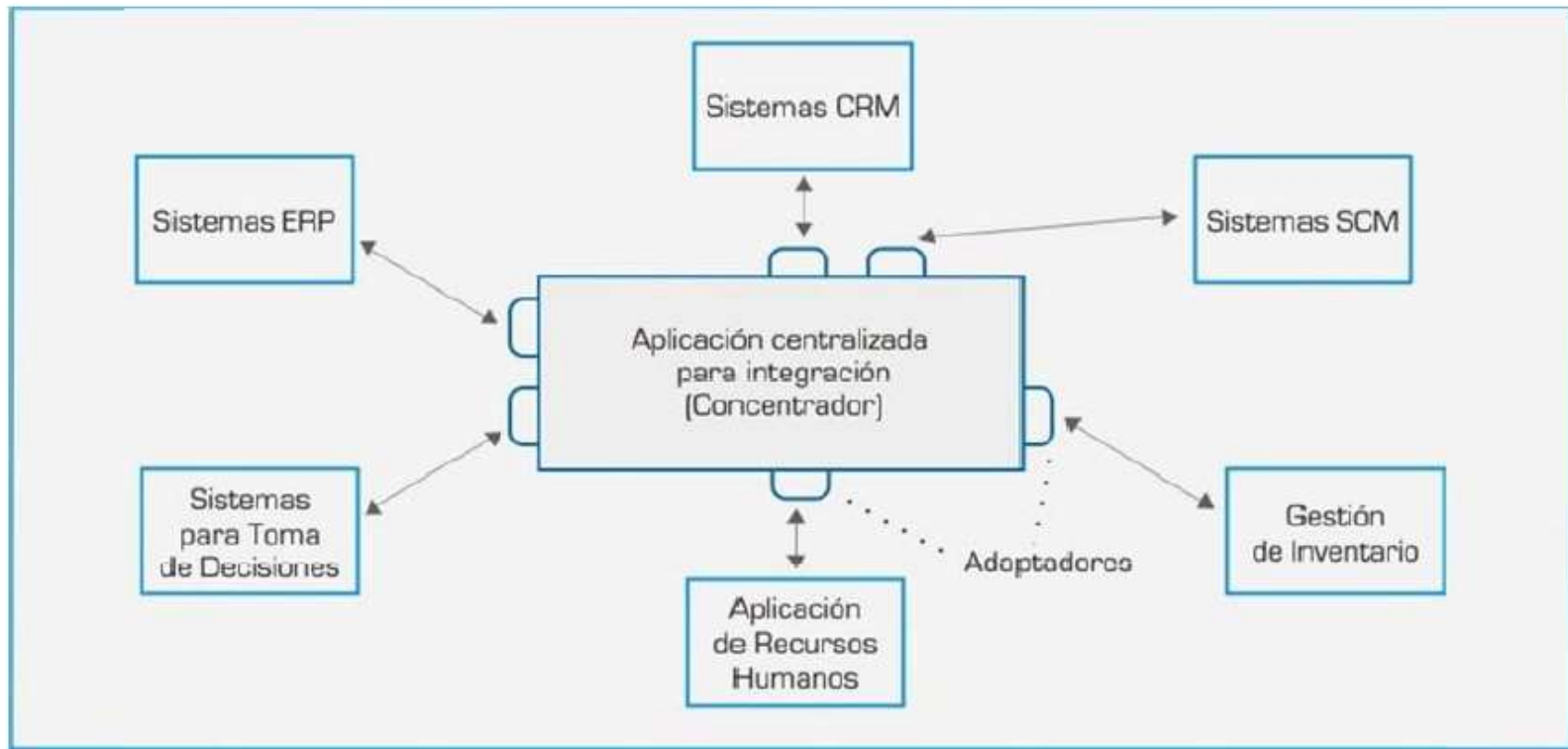
Mecanismo de Integración Punto a Punto



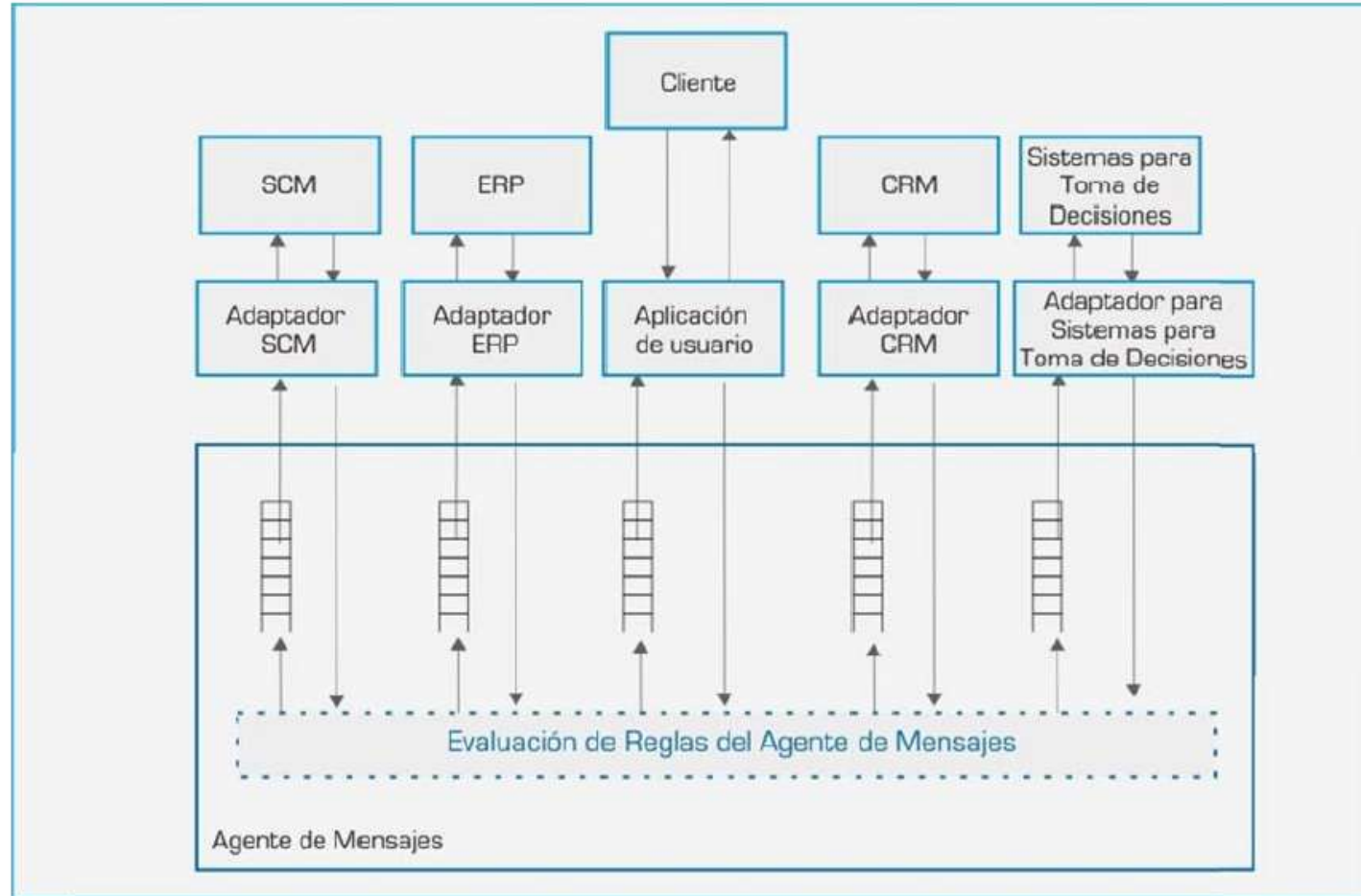
Inconvenientes de la integración punto a punto

- Arquitectura accidental (sincrónica, de grano fino y no escalable)
- Dificulta los cambios: nuevos canales, nuevos productos.
- Mantenimiento lento y caro
- Costoso de gestionar, monitorear y extender

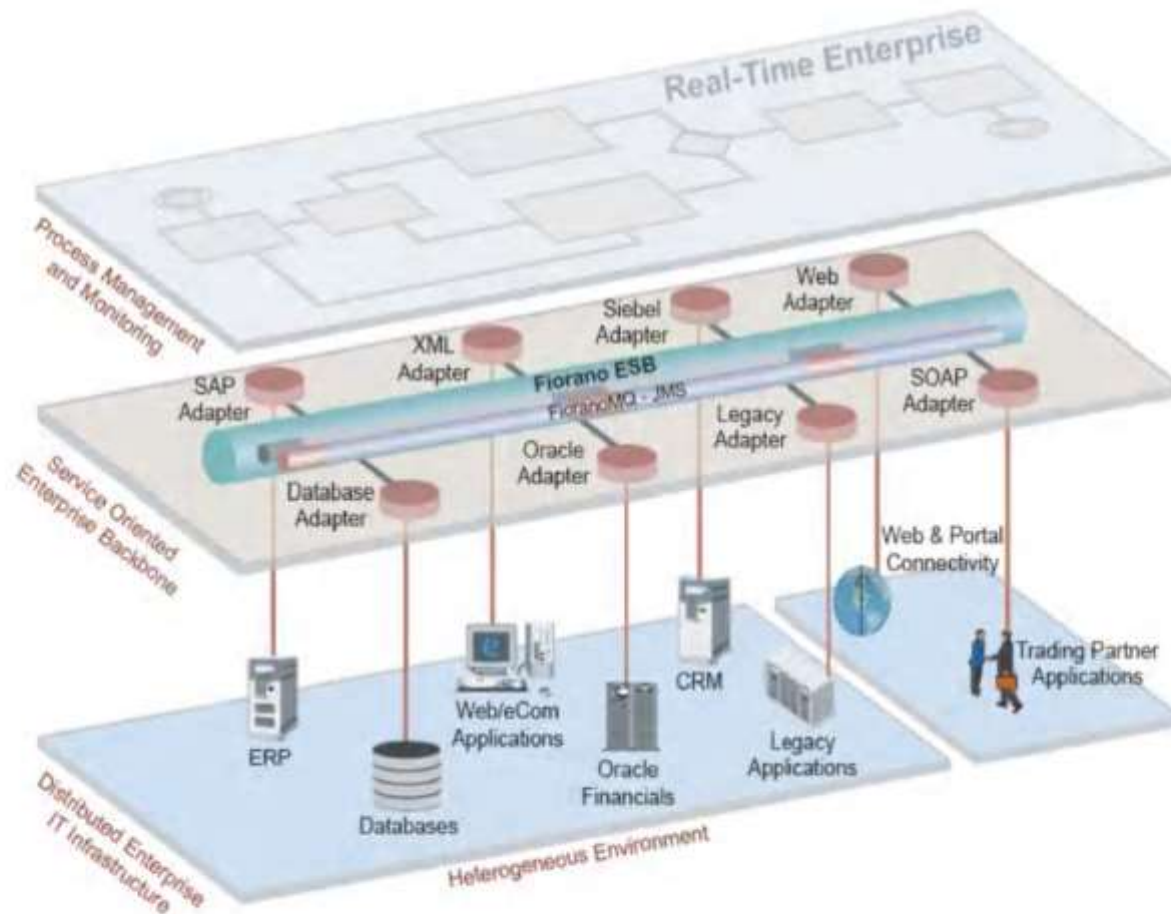
Mecanismo de Integración por Adaptadores



Mecanismo de Integración por Bus o Mediador



Orquestación de Servicios con Procesos de Negocio



Definiciones de Sistemas Distribuidos

- Tanenbaum:

A distributed system is a collection of independent computers that appears to its users as a single coherent system

- Colouris:

A distributed system is one in which components located at networked computers communicate and coordinate their actions only by passing messages

[Arquitectura de dos capas]

- Surge en los 90 como solución para potenciar el poder de las pc's y concentrar funcionalidad a nivel de "servidor"
- La arquitectura Cliente/Servidor constituyo uno de los primeros paradigmas "distribuidos"

[Ventajas del Cliente/Servidor]

- * Aumento del poder de las PC reduciendo costos
- * Permite que el procesamiento resida cerca de la fuente de datos reduciendo el trafico de red
- * Facilita el uso de GUI
- * Acepta el desafio de los sistemas abiertos
- * Favorece el diseño modular

[Desventajas del Cliente/Servidor]

- * Un desbalanceo de tareas entre cliente y servidor puede provocar cuellos de botella
- * El desarrollo de aplicaciones es mas complejo creciendo también la complejidad de las herramientas de desarrollo y programación

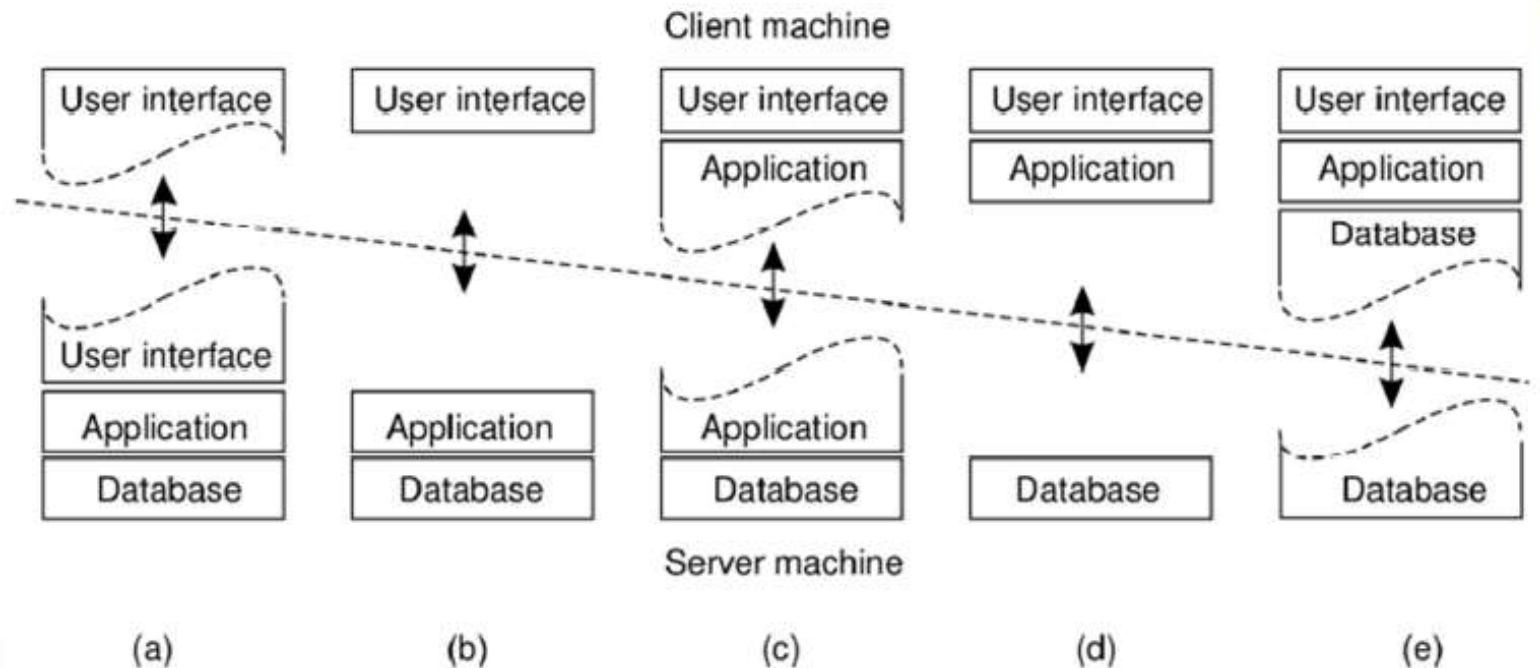
[¿Qué es Cliente/Servidor?]

- * Servicio
- * Recursos compartidos.
- * Protocolos asimétricos
- * Transparencia de la ubicación
- * Intercambios basados en mensajes
- * Encapsulamiento de servicios
- * Escalabilidad
- * Integridad

[Variantes de Cliente/Servidor]

- * File Servers
- * Database Servers
- * Transaction Servers
- * GroupWare Servers
- * Object Servers

[Aplicación en dos niveles]



[¿Que es MiddleWare?]

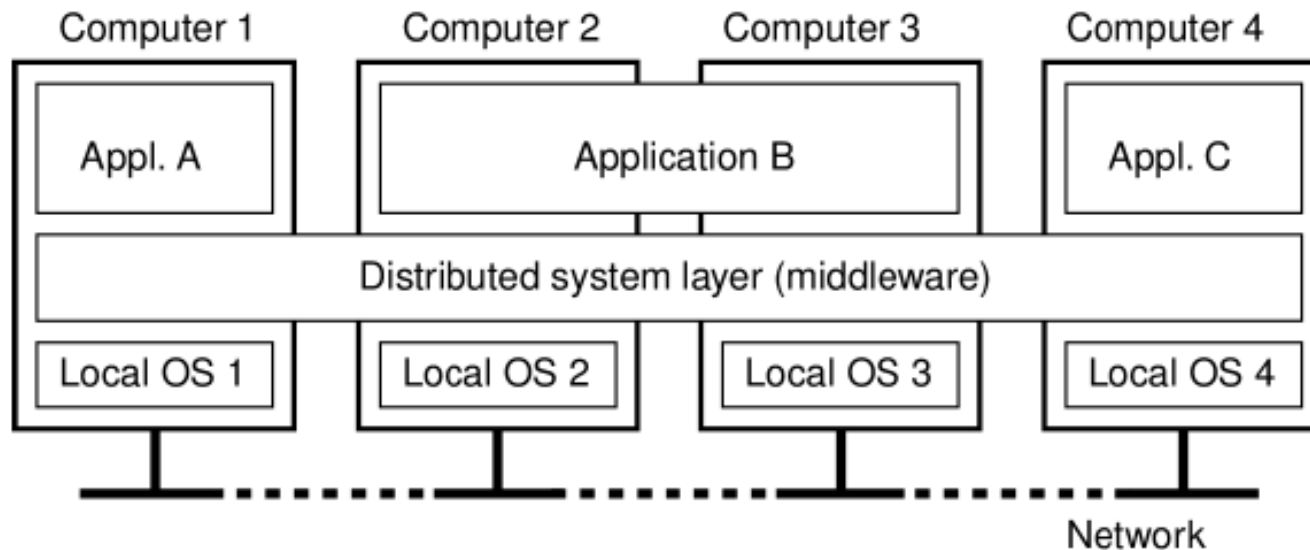
El MiddleWare comienza con la porción de cliente que es usada para invocar el servicio, cubre la transmisión sobre la red y retorna el resultado.

Existen dos grandes clases de middleware:

- General
- Especifico del servicio

Objetivo del Middleware

- Computadoras independientes
- Sistema único



[Middleware para Comunicación]



[Componente Middleware]

Corre en ambos lados de la aplicación, el lado del cliente y el del servidor.

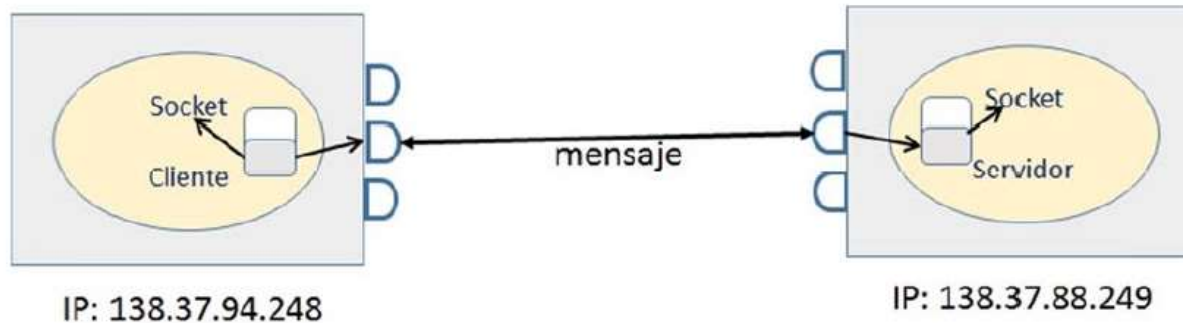
Dividimos esta componente en tres categorías:

1. Sistema operativo de red
2. Protocolo de transporte
3. Especifico del servicio

[Componente Middleware]



[Middleware por Socket]



Las operaciones de los socket son:

- 1- Se crea un socket con la función `socket`
- 2- Los clientes deben conectarse con `connect`
- 3- Los servidores aceptan conexiones de clientes usando la función `accept`
- 4- Clientes y servidores intercambian datos con `read` y `write`
- 5- Las conexiones se cierran con `close`

[Middleware específico del servicio]

El ambiente de programación RPC

El nacimiento del RPC corresponde al año 84 y especifica una manera diferente de trabajar sobre las redes que provoque el mismo efecto que se obtiene con la programación en un ambiente único con la idea de programa y subrutina. La idea nueva es obtener aplicaciones que funcionen exactamente igual en un ambiente centralizado que en un ambiente distribuido

[Características del RPC]

- **Pasaje de parámetros**
- **Binding**
- **Tratamiento de excepciones**
- **Trasparencia de la representación de datos**

[Mensajes y Colas]

- MOM (Message-Oriented Middleware) permite intercambio de mensajes de cualquier tipo entre cliente y servidor donde dichos mensajes son encolados. Las aplicaciones se comunican poniendo mensajes en una cola de mensajes.

[MOM. Características]

- Oculta totalmente los detalles físicos de la red y la comunicación se realiza sin saber si existe un enlace
- Los programas no hablan unos con otros en forma sincrónica.

MOM vs. RPC

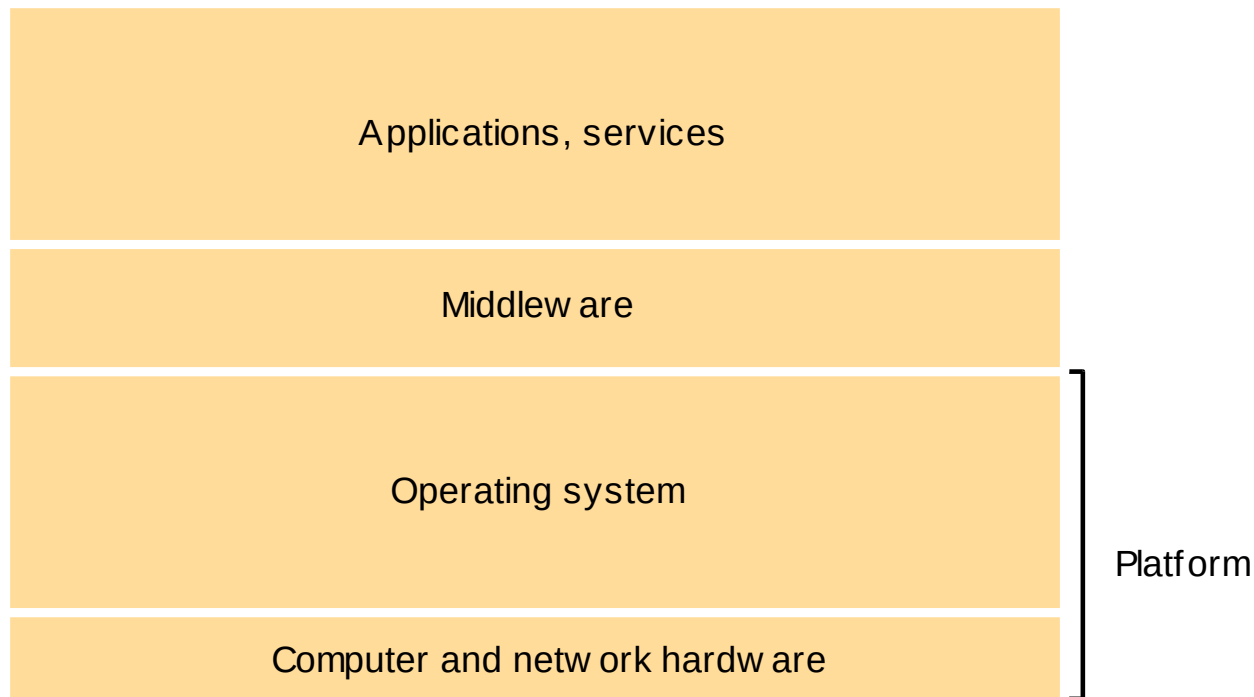
Característica	MOM	RPC
Metafora	Como oficina postal	Como telefono
Tiempo de relacion C/S	Asincronico. Clientes y servidores operan en distinto tiempo	Sincronico. Clientes y servidores corren a la vez y se esperan
Secuenciamiento C/S	Sin secuencia fija	Los servidores se lanzan antes que los clientes
Estilo	Cola	Call/Return
Datos persistentes	SI	NO
Compañero disponible	NO	SI
Balanceo de carga	Una cola simple FIFO o con prioridades	Requiere un monitor de TP adicional
Soporte transaccional	SI (algunos productos)	NO (requiere RPC transaccional)
Flitrado de mensajes	SI	NO
Performance	Baja	Alta
Procesamiento asincrónico	SI (se requiere colas y triggers)	Limitado (requiere hilos y trucos de programación)

[Arquitectura distribuida]

A grandes rasgos un arquitectura distribuida consiste de cuatro niveles:

- Nivel de aplicación
- Nivel de sistema operativo distribuido
- Nivel de manejo local
- Nivel del sistema de comunicación

[Niveles de la Arquitectura]



[El protocolo transporte del middleware]

APLICACIÓN	MOM	RPC		Peer-to-Peer
		Named Pipes	DCE	
PRESENTACION		NDR	SXDR	
SESION	NETBIOS / SOCKETS / TLI			
TRANSPORTE	RED	TCP/IP	IPX/SPX	
RED				
LLC		802.2		
MAC	TOKEN RING/ETHERNET		SDLC	
FISICA	FIBRA OPTICA	COAXIL	TELEFONO	

[Servicios de seguridad distribuidos]

El ambiente cliente/servidor introduce un nuevo tratamiento de la seguridad respecto del que se tenía en los sistemas tradicionales.

En un ambiente cliente/servidor no se puede asegurar que cualquier sistema operativo sobre la red proteja los recursos del servidor de acceso no autorizados y aunque se contara con clientes totalmente seguros, la red en sí es un medio altamente violable e inseguro.

[Nivel C2 de seguridad]

- **Autenticación:** (soy quien digo ser).
- **Autorización:** (sólo uso lo que me es permitido)
- **Auditoría:** (qué hice)

[Lecturas recomendadas]

- Libro de cátedra “Aplicaciones, servicios y procesos distribuidos. Una visión para la construcción de software”.
- “Distributed Systems Concepts and Designs” Fifth Edition 2012. George Colouris, Jean Dollimore, Tim Kindberg. Addison Wesley.
- “Distributed Systems Principles and Paradigms”. Andrew Tanenbaum, Maarten Van Steen. Pearson Edition. 2014.